Privacy-Preserving Shopping Intelligence via Federated and On-Device Learning

How much next-product recommendation quality survives when shopping data never leaves the device?

Group members
Ahmed Sherif
Eid Abdelrihem
Ahmed Abdelhamed

Supervised by
Prof. Rongxing Lu, Queen's University
Prof. Mona Farouk, Cairo University
Prof. Motaz El-Saban, Microsoft (industry mentor)

MSc in Artificial Intelligence and Data Science
School of Computing, Queen's University · October 20, 2026



Grant Hall, Queen's University. Photo: Takipoint123, CC BY-SA 4.0, Wikimedia Commons

The research question

“How much shopping recommendation quality can be retained when moving from cloud-based to on-device or federated models?”

Why it matters

- A browser (Microsoft Edge) sees shopping behaviour on the device
- Cloud training needs that history on a server, which users and regulators resist
- Most federated-recommendation papers use small datasets and sampled metrics, not real shopping logs

What we answer

- **Retention**: cloud → federated → on-device
- **Value of server pretraining** (warm start) and **on-device adaptation**
- **Privacy**: cost of DP, leakage audits
- **Deployability**: INT8, latency, upload bytes

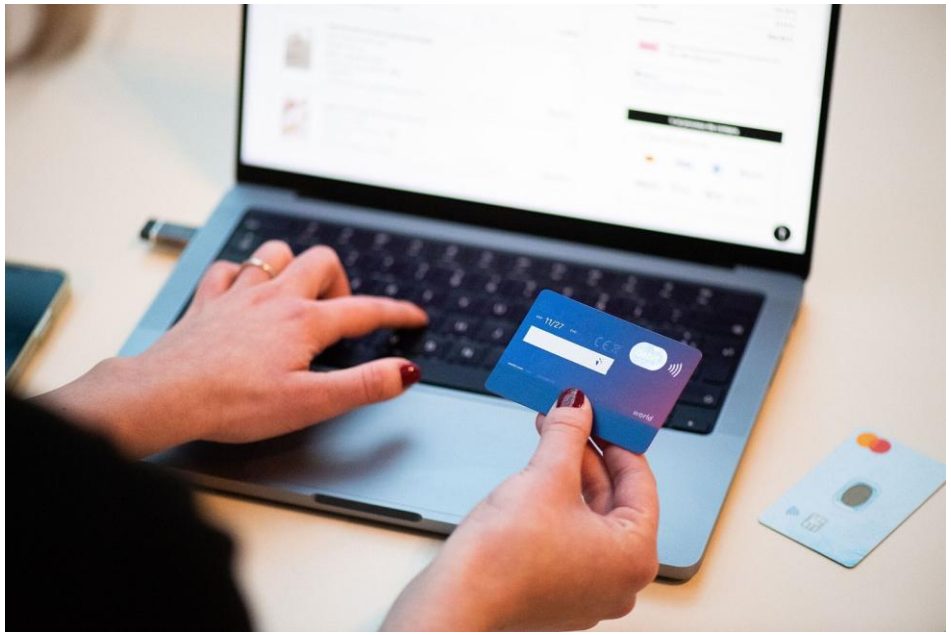
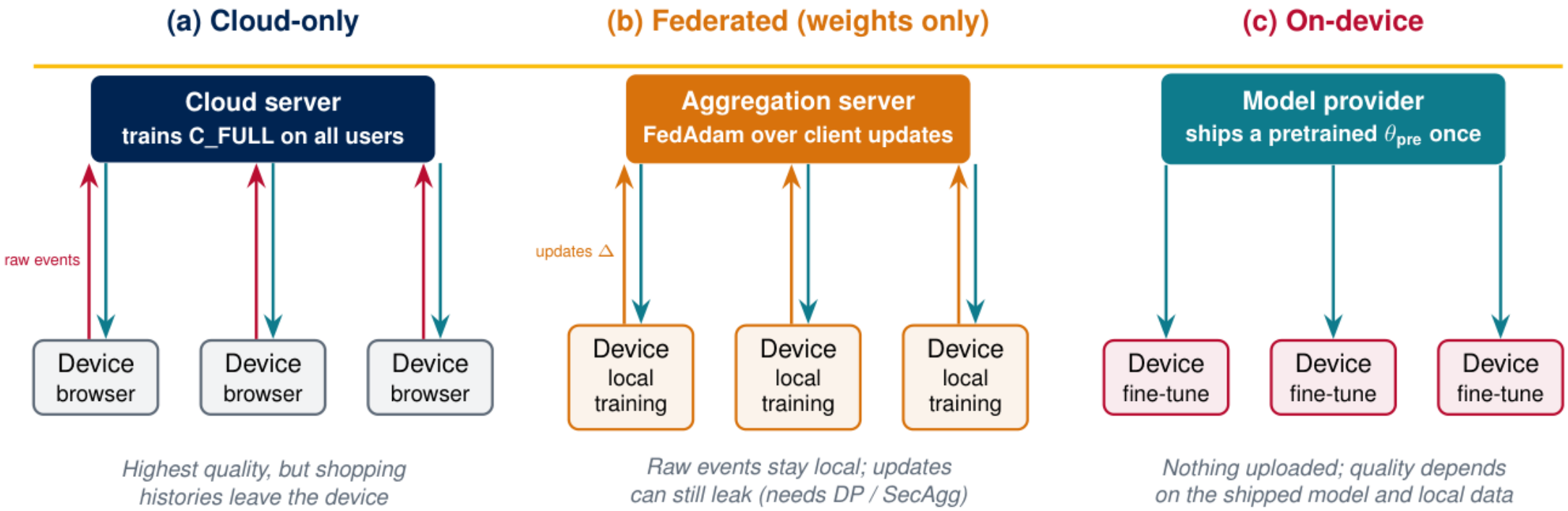


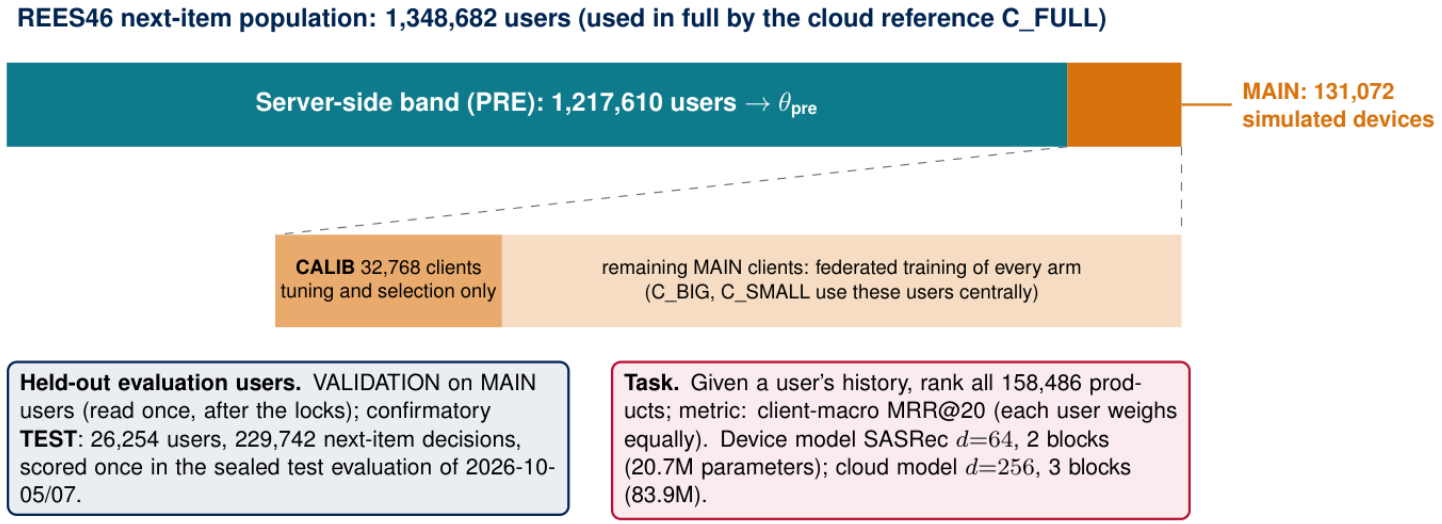
Photo: Bogdan Hoyaux / European Commission, CC BY 4.0, Wikimedia Commons

Three ways to deploy a shopping recommender

Every comparison in this project is between these three architectures



How we built a rigorous answer

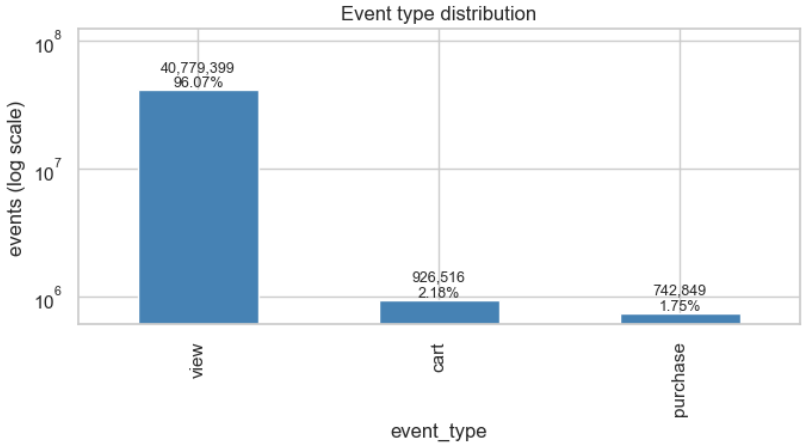


Design choices

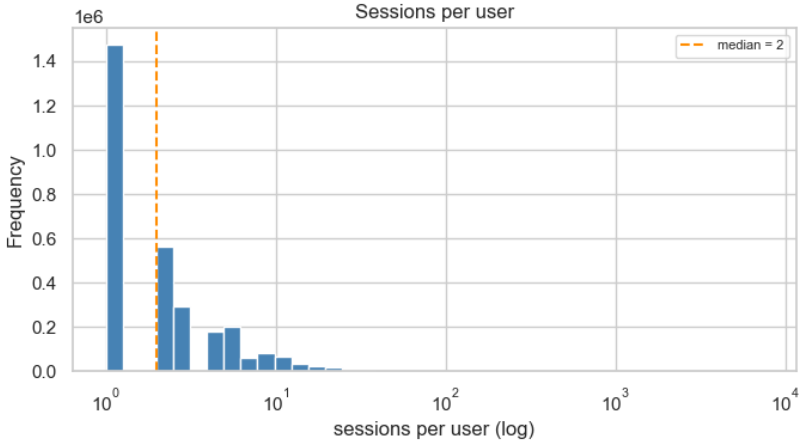
- REES46 e-commerce log, Oct–Nov 2019
- Rank the **full catalogue** (158,486 products) — no sampled negatives
- Client-macro **MRR@20**; retention = model / cloud reference
- SASRec: device d=64 (20.7M) · cloud d=256 (83.9M)
- **Pre-registered**: every decision locked before **one** TEST look
- 6 public benchmarks as a second check

The data: real shopping sessions from one store

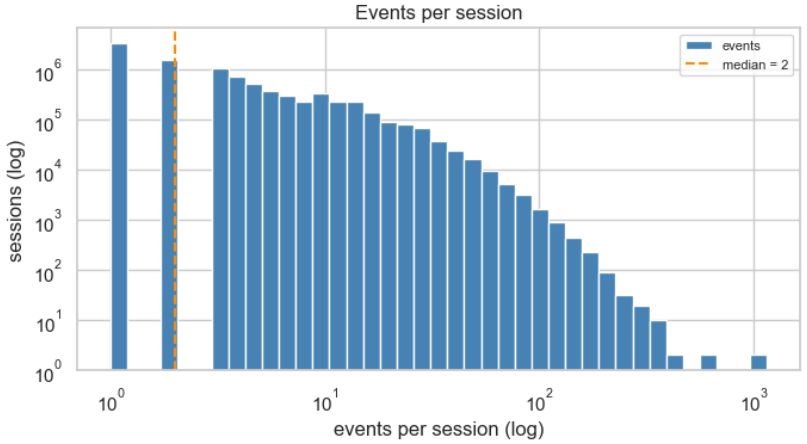
REES46, October 2019 month, as plotted by our data-audit notebook



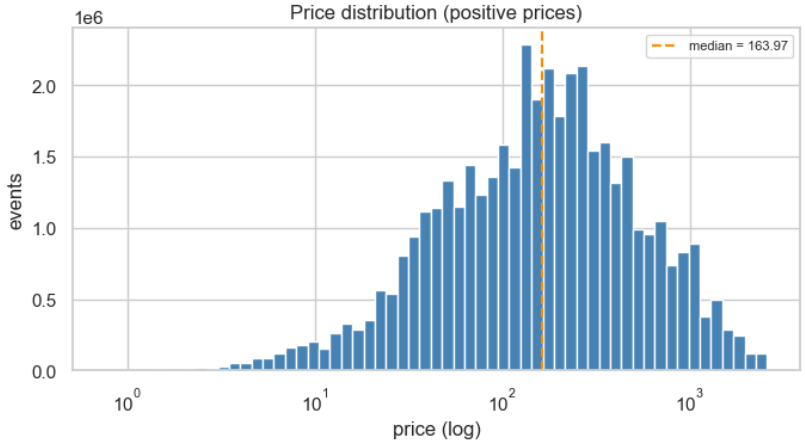
Event types: 96% views, 2.2% carts, 1.8% purchases



Sessions per user: median 2



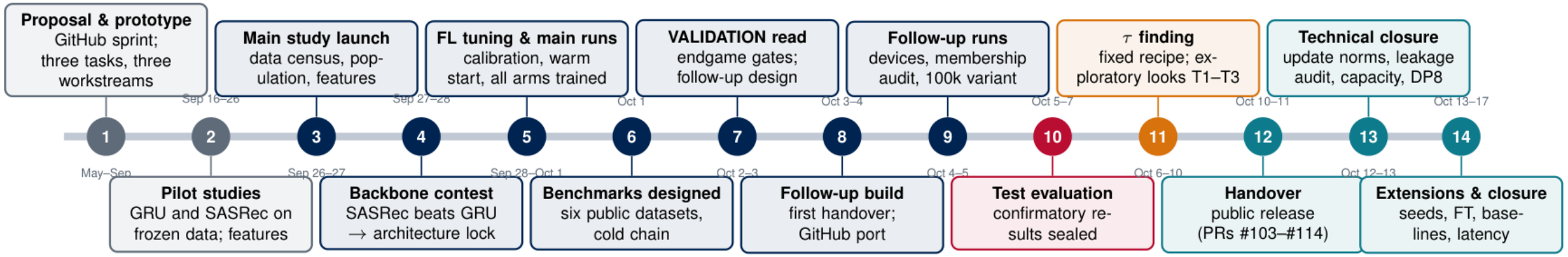
Events per session: heavy-tailed, median 2



Product prices: median 163.97

The research journey: 14 phases in five months

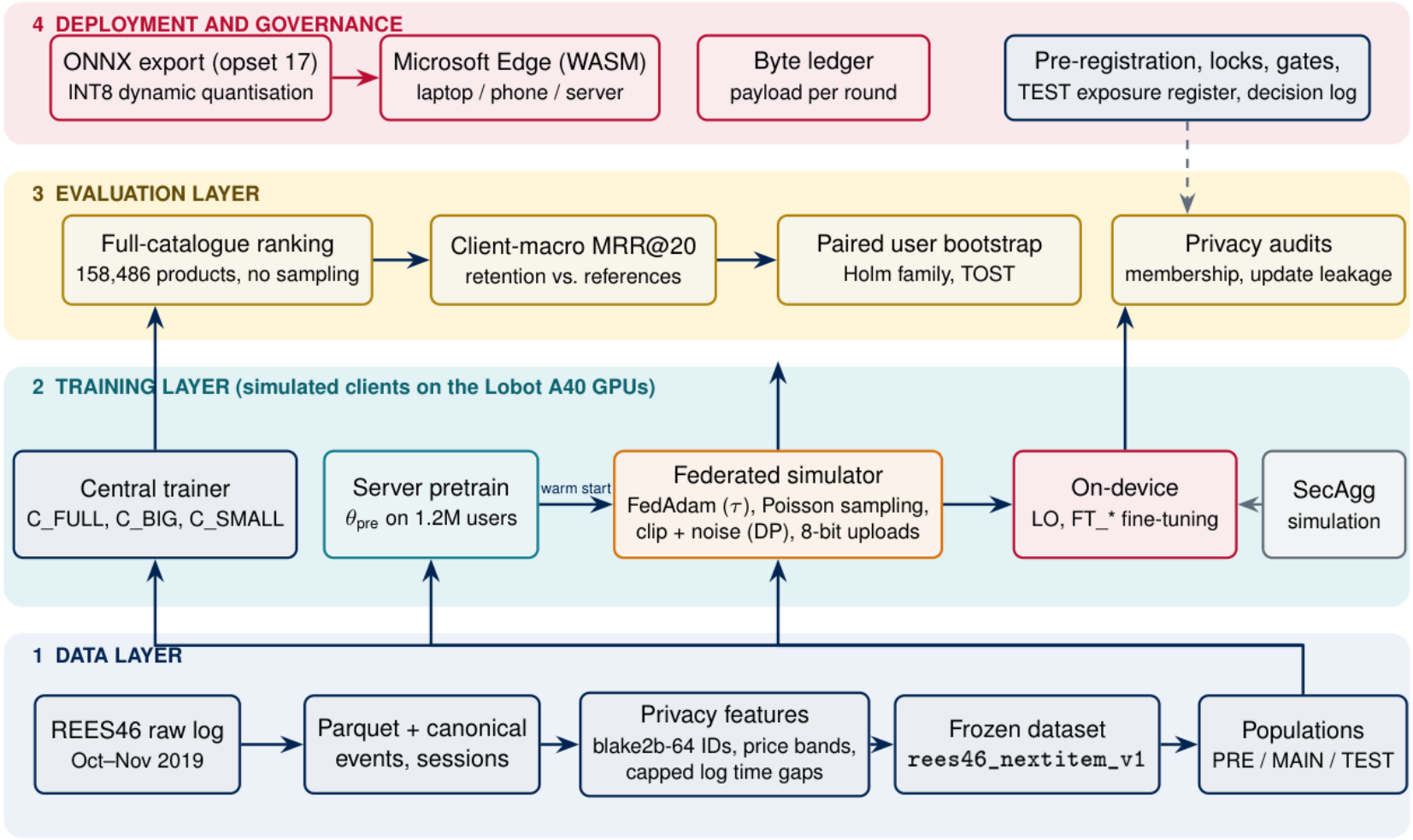
From the proposal to the final consolidation



Each phase answered a question raised by the previous one: prototype → stronger backbone → tuned federated recipe → sealed confirmatory test → diagnosis of why federation added nothing → fixed recipe, privacy and device studies → closure.

System architecture

Data → training → evaluation → deployment and governance



The prototype stage and the system workstream

Prototype stage (Aug–Sep 2026)

- Raw October log: 42.4M events → Parquet → canonical events
- Cohort of 388,789 clients; sealed TEST window
- Three tasks (next category, purchase, re-rank)
- The GRU model beat the transition-table baseline
- Federated plumbing with Flower; pilot retention **88.38%** (validation)
- Lesson: a label defect and evaluator conventions → need a stronger protocol

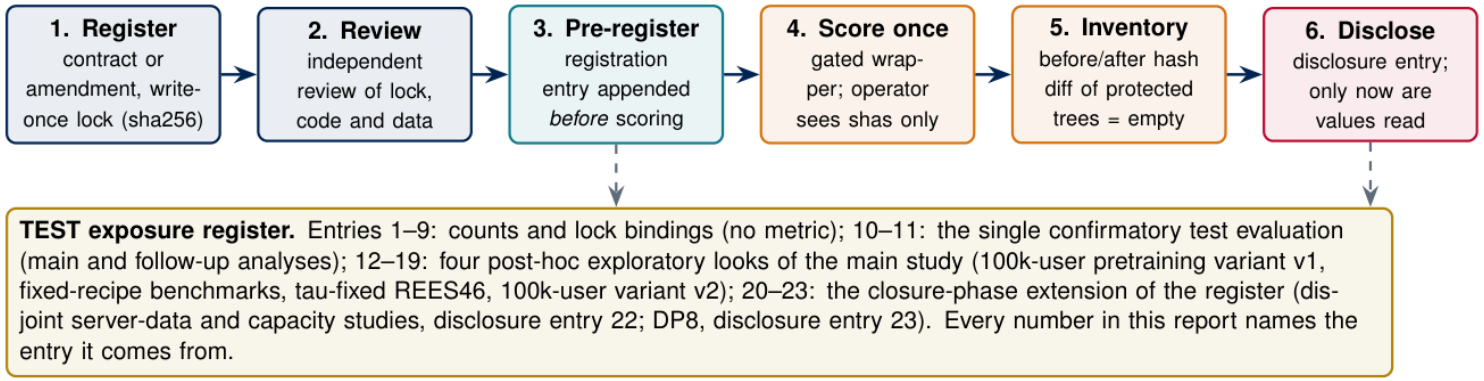
System, deployment & security workstream

- blake2b-64 hashed identifiers; price bands; capped log time gaps
- ONNX export (opset 17) with a dynamic-batch fix
- INT8 dynamic quantisation + benchmark
- Communication byte ledger per round
- Quality-retention registry
- Threat model, secret handling, CI with a lock-file check

Main study: backbone contest and the gated test protocol

Backbone contest (validation)

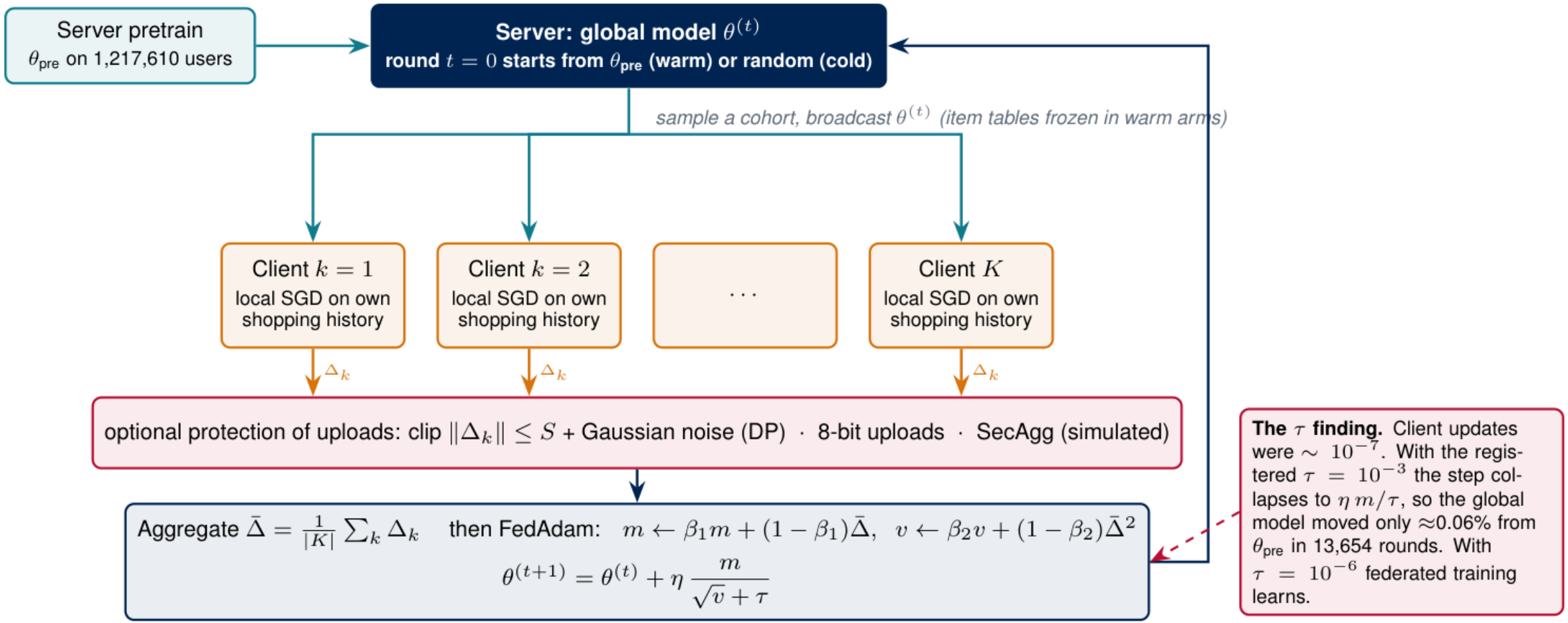
- GRU family mean 0.1569
- SASRec family mean 0.1612
- Outcome DECISIVE_STABLE → architecture lock on SASRec



Why so much protocol?

- 33 main-analysis contrasts, a Holm family of 18, two seeds, many arms — easy to fool ourselves
- Selection only on calibration data; main-population validation read once; TEST read once in a sealed evaluation
- Every later exploratory test look is a numbered, disclosed register entry (12–23)

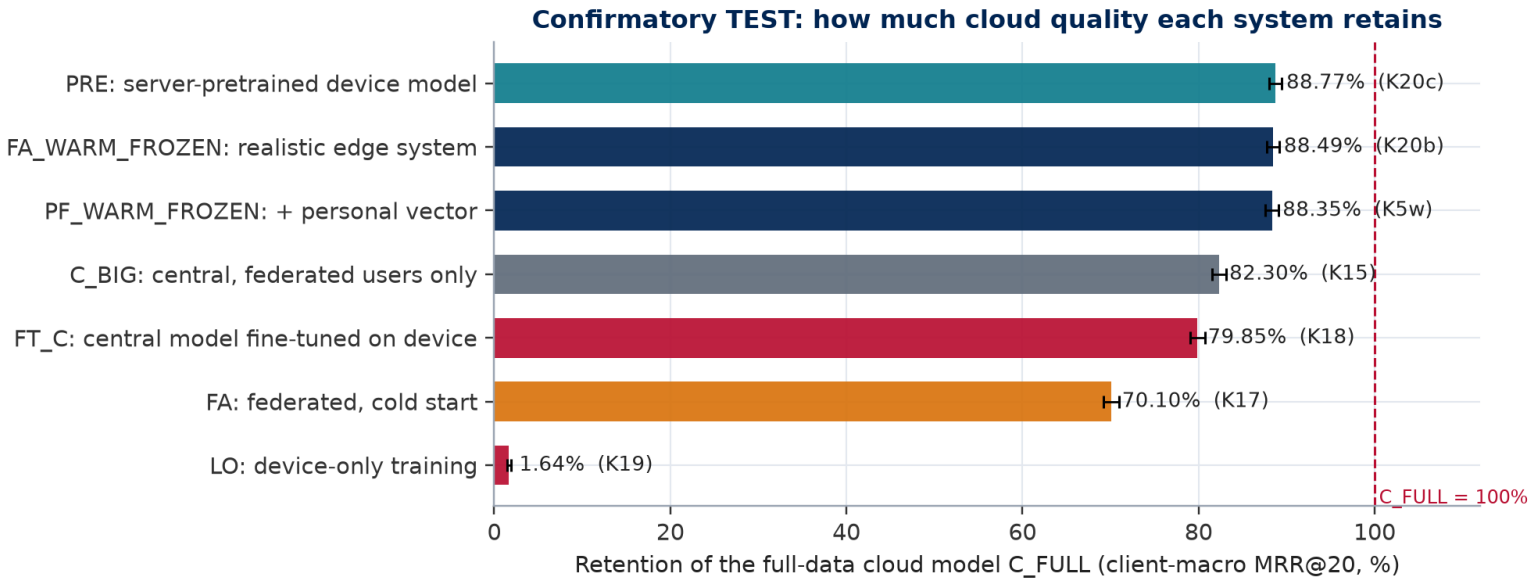
The federated recipe: warm-started FedAdam



Confirmatory answer: how much cloud quality survives

REES46 TEST, 26,254 users, client-macro MRR@20

(confirmatory result)

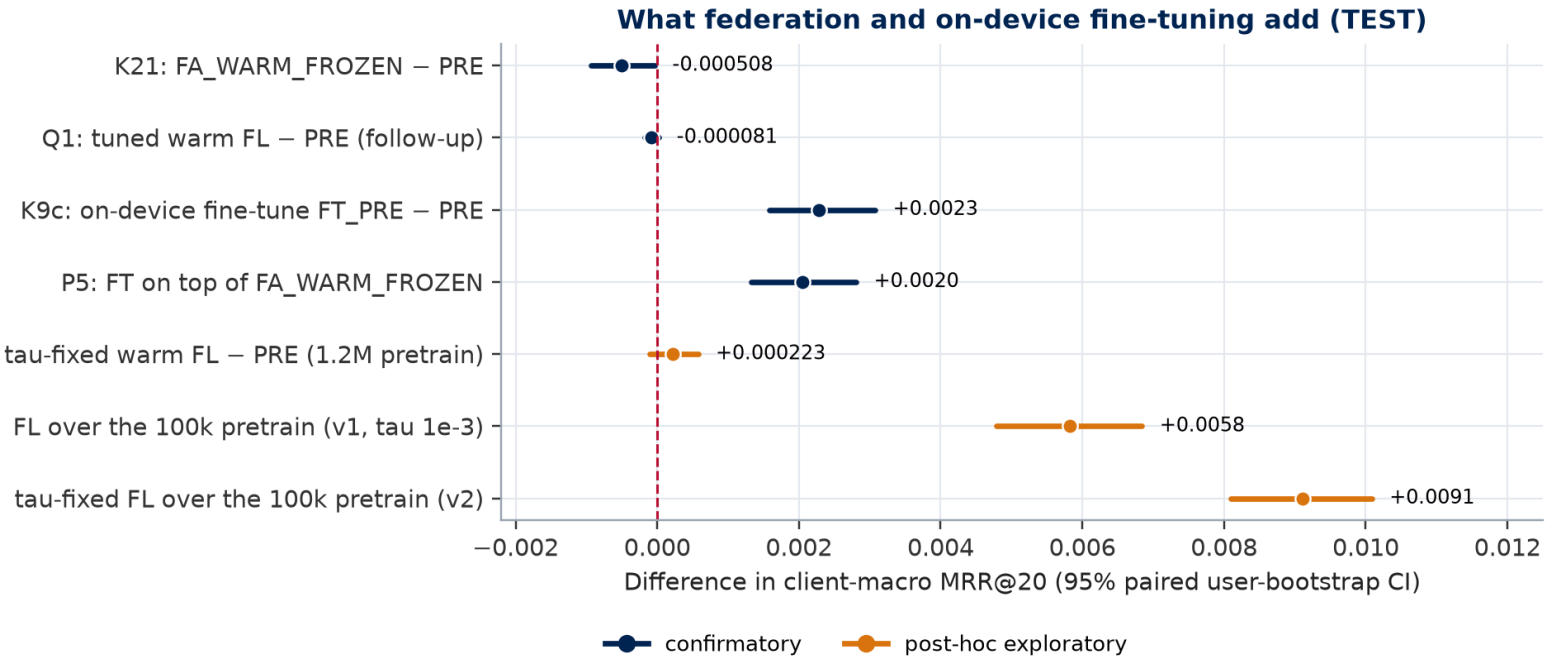


Key numbers

- Realistic edge system (warm FL, frozen tables): **88.49%** [87.73, 89.18]
- Pretrained model alone: **88.77%**
- Cold federated: 70.10%
- Device-only training: 1.64% of C_FULL
- Warm start over cold FL: +0.0331 MRR@20

What federation and fine-tuning add

Differences in MRR@20 with 95% paired-bootstrap CIs

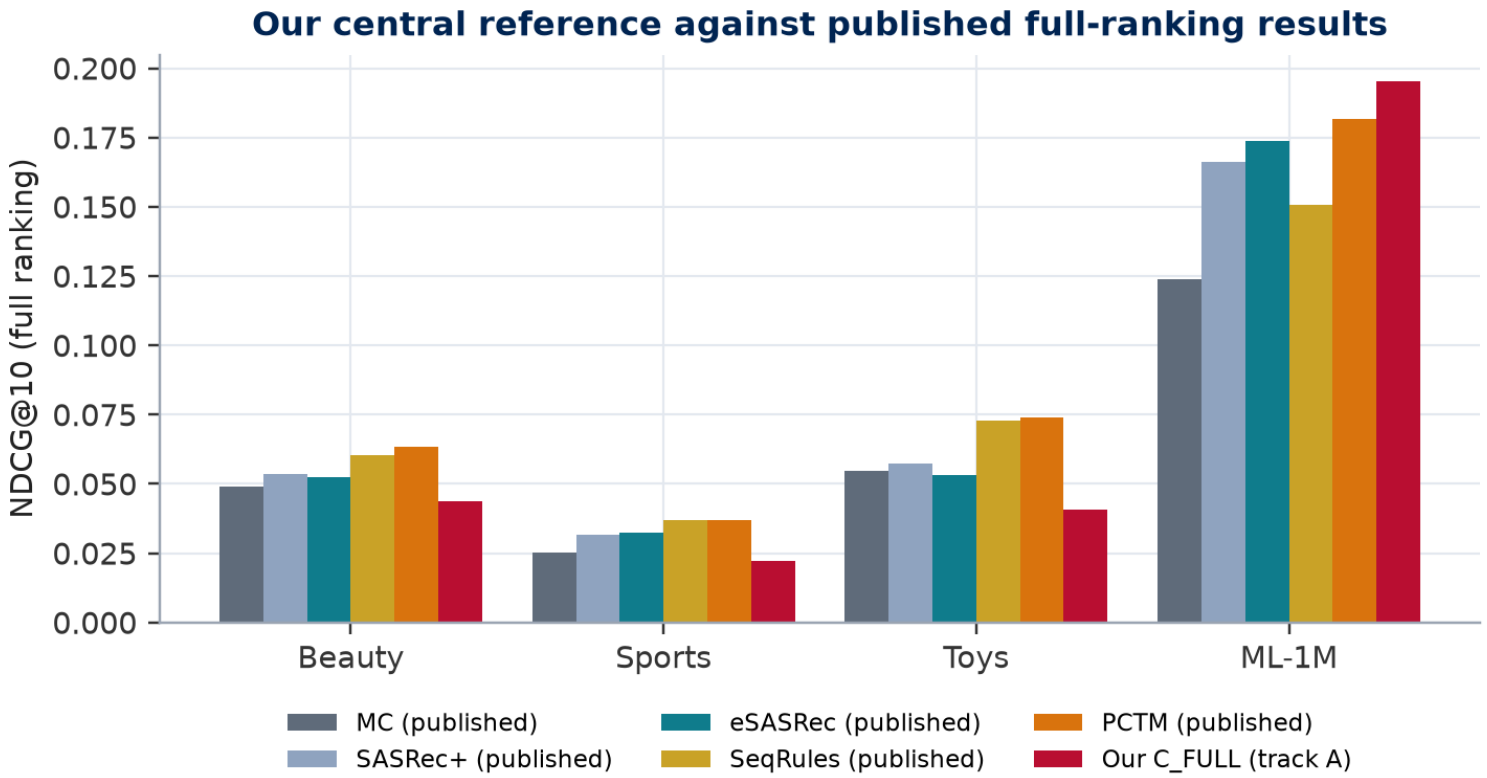


Reading

- **K21** FL – PRE: −0.000508 → federation adds nothing over a strong pretrain
- **Q1** tuned warm FL − PRE: EQUIVALENT (±0.0005)
- **K9c** on-device fine-tune: +0.002283 — the deployable gain
- Orange: post-hoc looks (next slides)

Benchmarks: our cloud reference vs published results

Full-catalogue NDCG@10 on public datasets

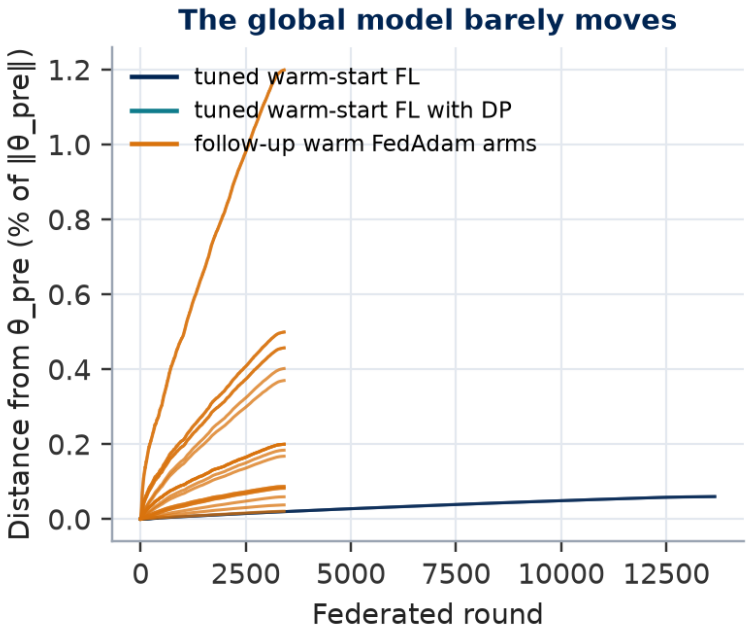


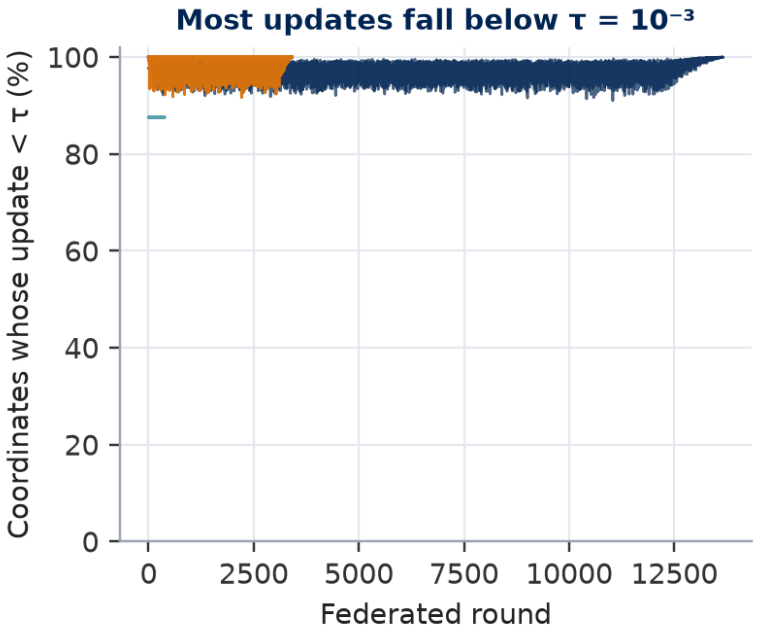
Confirmatory benchmark looks

- C_FULL wins on every dataset; every FL arm far below it
- ML-1M: C_FULL 0.1936 vs published eSASRec 0.1739
- Confirmatory FL retention only 13–77%
- → something in the federated recipe was wrong

Diagnosis: FedAdam's τ silently froze federated training

(exploratory result)





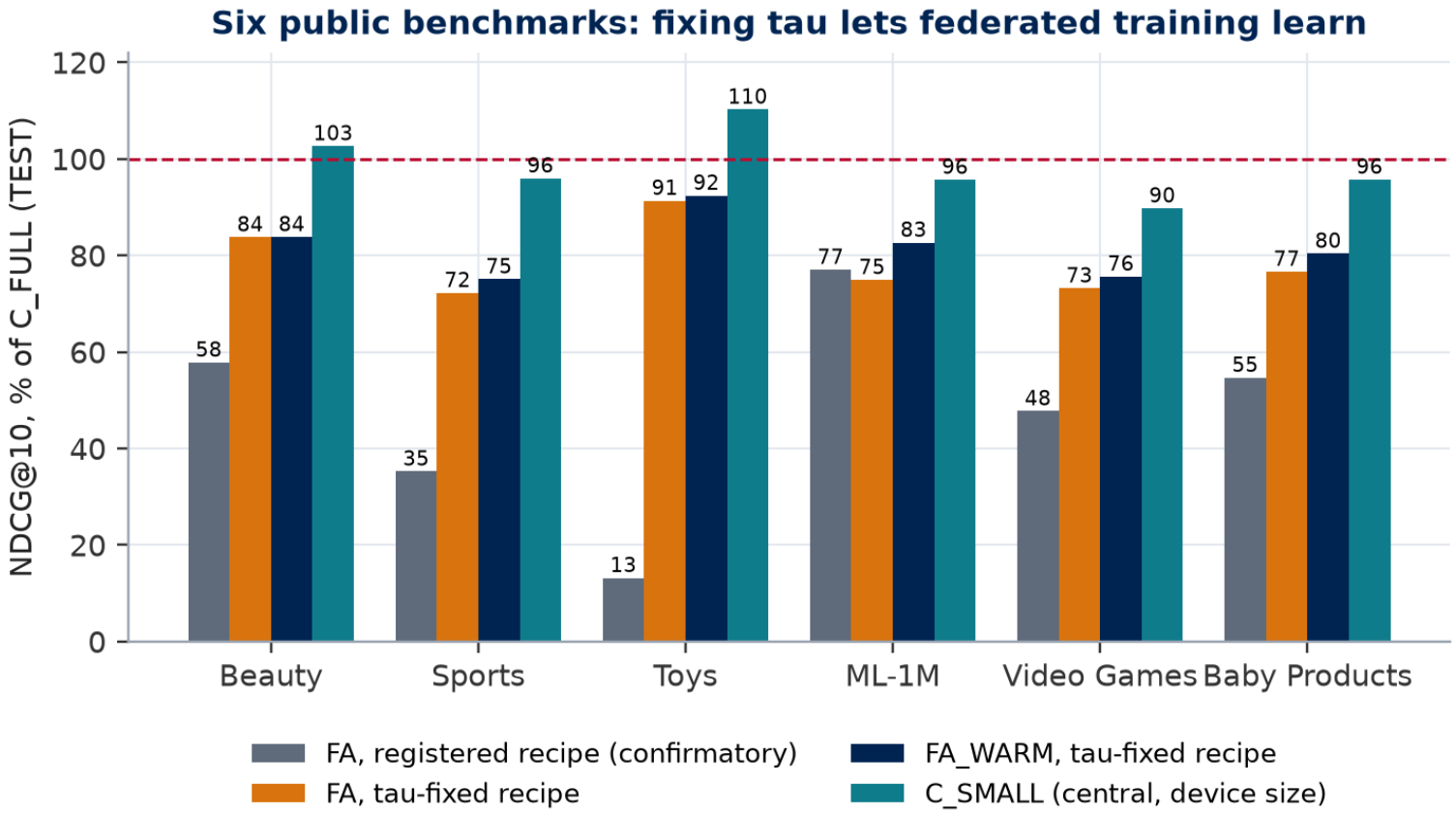
Mechanism

- FedAdam step ∝ m / (√v + τ)
- Client updates ≈ 10^{-7}, registered τ = 10^{-3}
- → server step 20–100× too small
- Tuned model moved ≈ 0.06% from θ_pre in 13,654 rounds
- Confirmatory results stay as registered; the fix is exploratory

With τ fixed, federated training learns

Six public benchmarks, TEST (exploratory look)

(exploratory result)

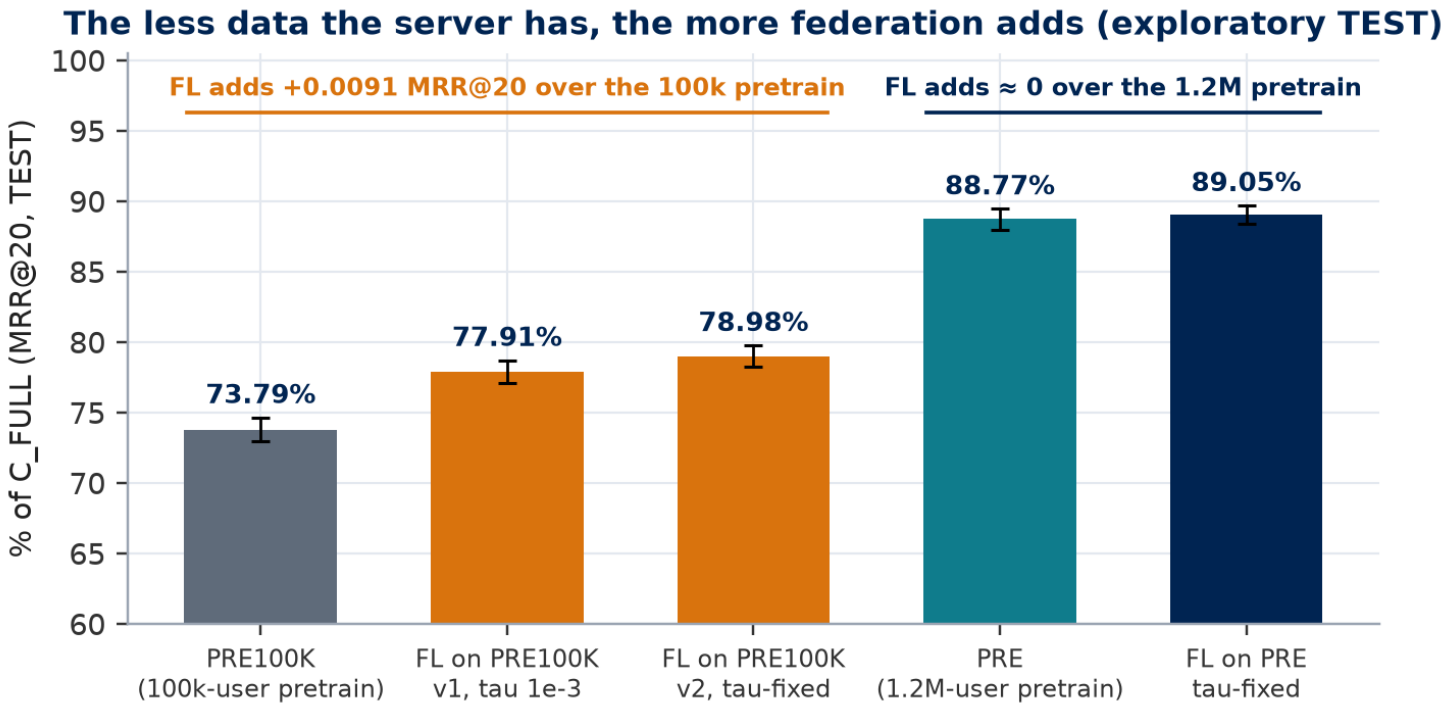


What changed

- Warm FL: 75–92% of C_FULL (was 13–77%)
- C_SMALL (device size, central) 90–110%: size is not the bottleneck
- On REES46 the fix changes nothing (+0.00022, CI includes 0)
- Caveat: τ changed together with budget / item tables

The value of federation depends on the server's data

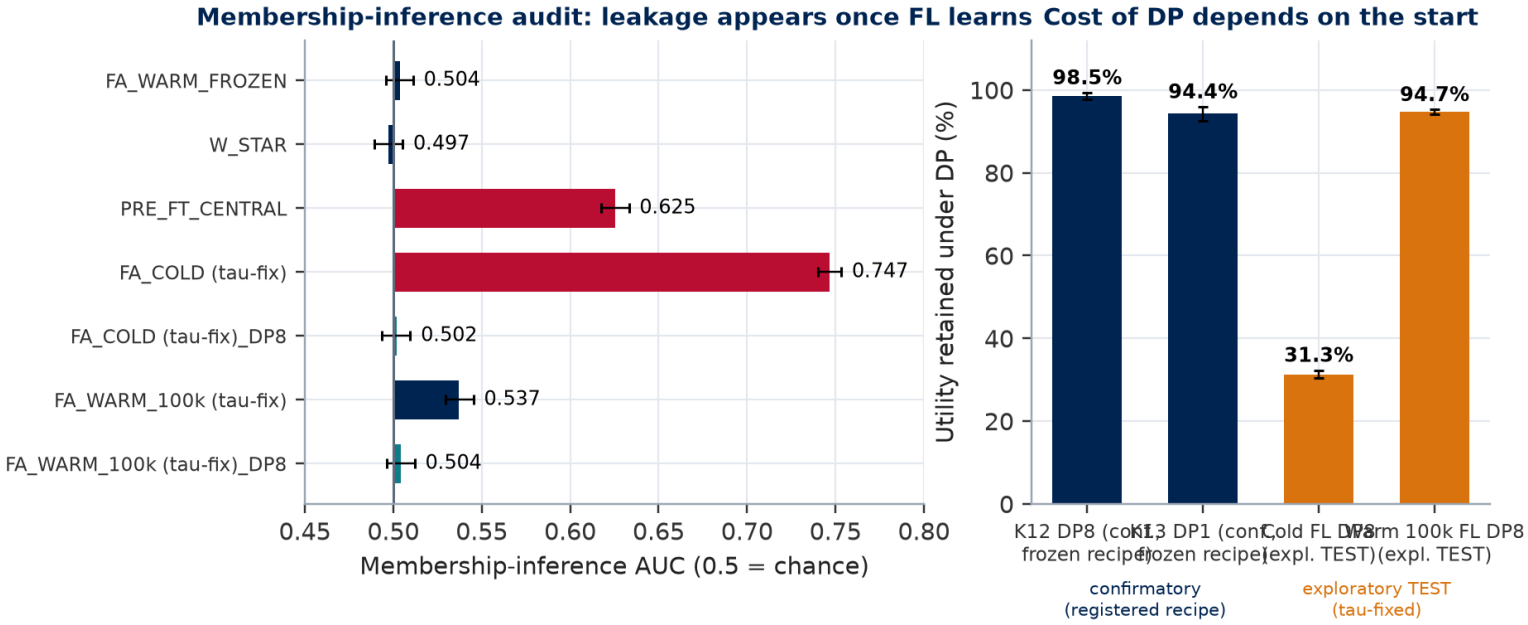
(exploratory result)



Evidence

- 100k-user pretrain: FL adds **+0.0091** MRR@20
- 1.2M-user pretrain: FL adds ≈ 0
- Disjoint server data (50%): warm beats cold by 27–55% on Amazon data
- Exception: MovieLens-1M (6,040 users)
- Federation cost at matched size ≈ 14–22%

Privacy: learning leaks, and the guarantees need care

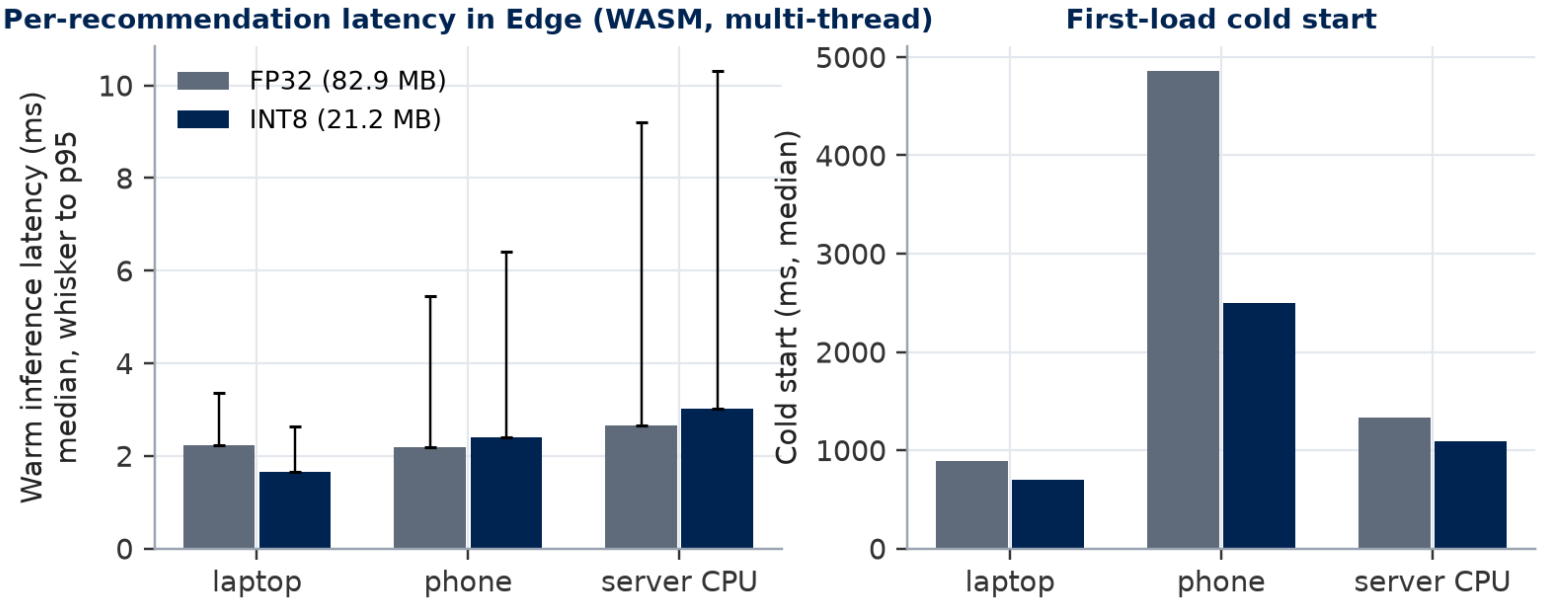


Findings

- FL that really learns leaks membership (AUC 0.747)
- DP at ε = 8 removes the measured leak; cost small on warm, large on cold
- **Audit corrections:** legacy DP seed was public → no formal DP guarantee
- Update-leakage attack: 2.625 pp > 2 pp threshold (negative)
- Secure aggregation: simulation only (< 1% upload overhead)

On-device feasibility in Microsoft Edge

ONNX Runtime WebAssembly, laptop / phone / server CPU

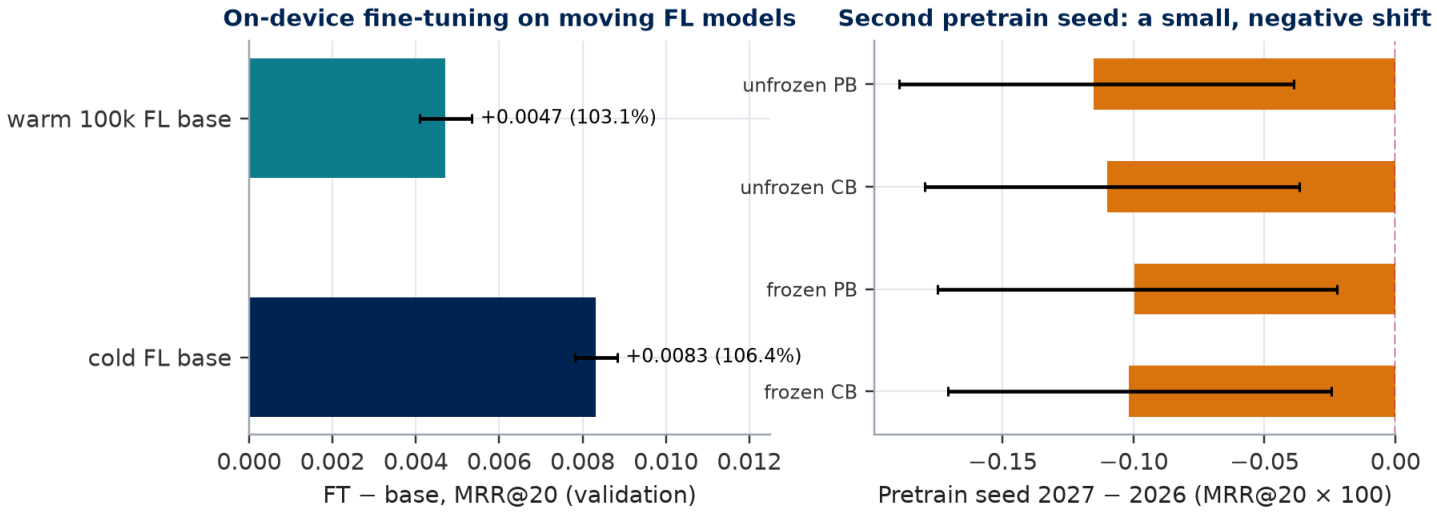


Deployability

- INT8 model: 82.9 MB → 21.2 MB
- INT8 and 8-bit uploads: no measurable quality cost (K16, K14)
- Warm inference ≈ 1.7–3 ms per recommendation
- Phone cold start ≈ 2.5 s with INT8

Closure experiments

Technical closure and extensions, 12–17 October



What closure added

- Second pretrain seed: small negative shift (−0.10 pts)
- Fine-tuning moving FL models: +3–6%
- Training-free baselines: best MC 0.1366 (validation)
- Stopped / not run: DP-FTRL, canary restart, TEST2
- All results labelled; nothing re-scored

What we learned

What worked

- Warm start from a server pretrain: biggest single gain
- On-device fine-tuning: small, significant, cheap
- INT8 / 8-bit uploads: free compression
- Pre-registration: kept the main answer honest
- Full-catalogue evaluation: comparable with published work

What did not

- A badly scaled τ froze FL — invisible in the protocol
- FL over a strong pretrain: no added value
- Device-only training: unusable
- Legacy DP randomness: no formal guarantee
- Several studies stopped for time (DP-FTRL, TEST2)

Conclusion and future work

Answer

- A warm-started on-device/federated system retains **88.49%** of the cloud model
- Whether federation pays off depends on the server-side pretrain and a correctly scaled optimiser
- Fine-tune on the device; federate when the server knows little
- Privacy must be engineered and audited, not assumed

Future work

- Distil a large teacher into the device model
- DP with secret randomness at full scale
- Real secure aggregation
- December-2019 replication (TEST2)
- Real-device field trial in Edge

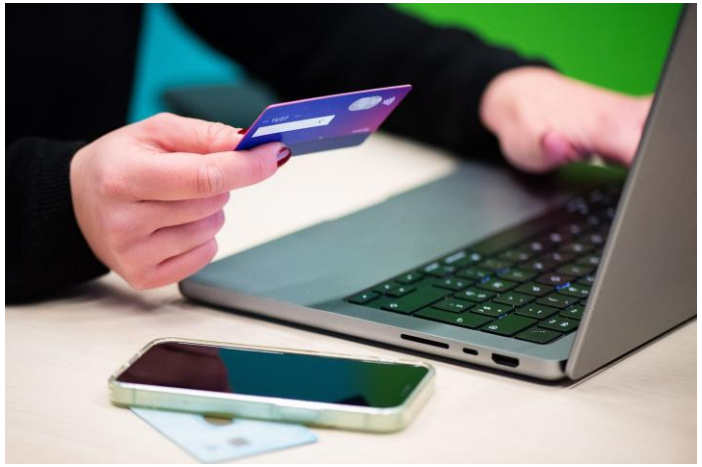


Photo: European Commission, CC BY 4.0, Wikimedia Commons

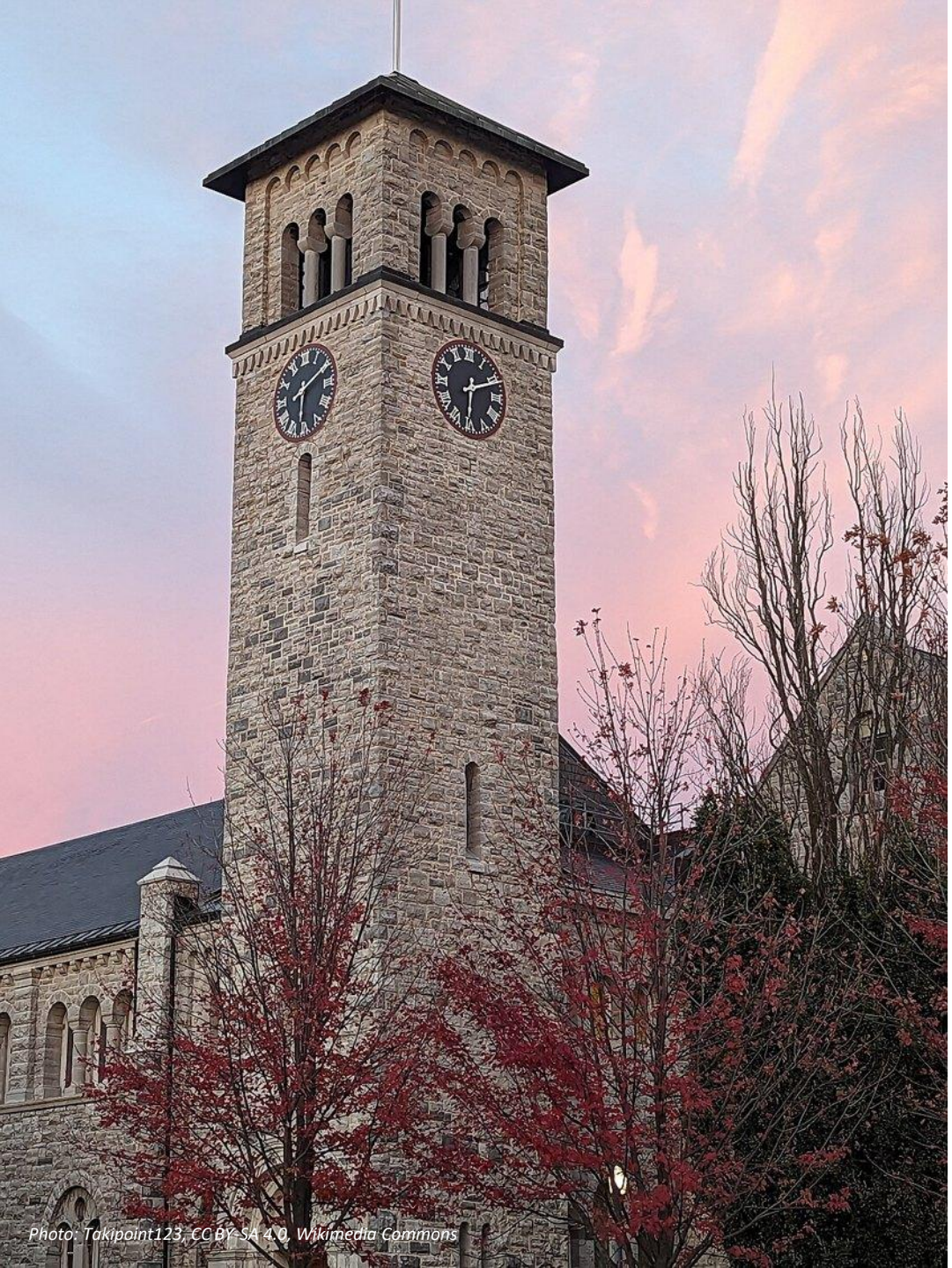


Photo: Takipoint123, CC BY-SA 4.0, Wikimedia Commons

Thank you

Questions and discussion

Digilians 2026, Group 8, Queen's University
Code: github.com/microsoftproject26/
Privacy-Preserving-Shopping-Intelligence